

1. Data Description and Pre-Processing

As we have seen the dataset in the previous semester and reported it with details in our MLC assignment, we will try not to delve too much into detail about the raw data, and will try to keep this section brief.

This dataset has near 402,000 rows and entries about car sale adverts from AutoTrade. It contains some details and features which related to different cars such as mileage, fuel type, body type, colour, price and etc.

At first we load the dataset and by using info function (data.info()), we display the details and information and type of each features.

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 402005 entries, 0 to 402004
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   public_reference       402005 non-null  int64
1   mileage                401878 non-null  float64
2   reg_code               370148 non-null  object
3   standard_colour        396627 non-null  object
4   standard_make          402005 non-null  object
5   standard_model         402005 non-null  object
6   vehicle_condition      402005 non-null  object
7   year_of_registration   368694 non-null  float64
8   price                  402005 non-null  int64
9   body_type              401168 non-null  object
10  crossover_car_and_van  402005 non-null  bool
11  fuel_type              401404 non-null  object
dtypes: bool(1), float64(2), int64(2), object(7)
memory usage: 34.1+ MB
```

By using filtering, unusual and incorrect data are removed so that more accurate analysis and training can be provided for this dataset. I filter the dataset to only show the data after 2000 in year of registration and the data before 200000 in mileage to make the dataset more logical.

```
# filtering the dataset, I decided to remove unusual data which you can see in previous plots
data = data[(data['year_of_registration'] >= 2000)]
data = data[data['mileage'] <= 200000]
```

You can see these changes in figure 1.1.

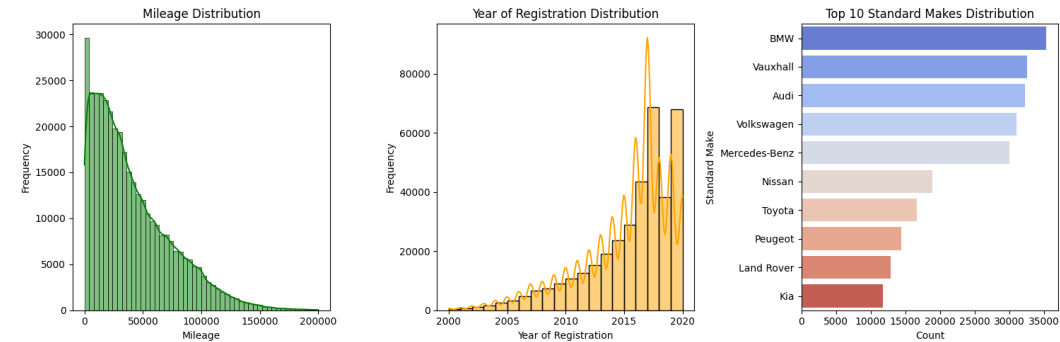


Figure 1.1

In this section, I also compared 3 main features with price. They are powerful to detecting the target which is price. (figure 1.2)



Figure 1.2

The scatter plot [Mileage Vs Price](#) explains a visible trend in the decrease of prices with increasing mileage. After a period of general price stability in the first few years, prices started rising in 2010. As you can see we can conclude that the newer the car, the more expensive it is (the middle plot). The box plot [Price distribution by Standard make](#) describes the way car prices are distributed among different brands.

Now I would like to talk about the data preprocessing. At first, I drop two features from our dataset which do not help us in target prediction. These two features are [public reference](#) and [crossover car and van](#) which are unnecessary.

```
data.drop(['public_reference', 'crossover_car_and_van'], axis=1, inplace=True)
```

The dataset was grouped into categorical variables and numerical variables. It created the opportunity of applying different preprocessing techniques for each of them.

Categorical Features: Handling missing values in categorical features was done with SimpleImputer more frequent value being imputed. Such strategy is helpful in maintaining the structure of categorical data.

Year of Registration: An additional custom function, `extract_year_from_code`, was introduced to predict year_of_registration from `reg_code` column. Thus, missing values were replaced by attributes, which were approximated during the calculation process, and the created interim column was deleted.

```
def extract_year_from_code(reg_code):
    if pd.isna(reg_code):
        return None
    try:
        # extract the age identifier from the registration code
        age_identifier = int(reg_code[:2])
        # determine the year based on the age identifier
        if age_identifier >= 50:
            year = 2000 + (age_identifier - 50)
        else:
            year = 2000 + age_identifier
    except ValueError:
        return None
    return year

# apply the function to the reg_code column to create a new year column
data['calculated_year'] = data['reg_code'].apply(extract_year_from_code)

# fill in missing values in year_of_registration with the values from calculated_year
data['year_of_registration'].fillna(data['calculated_year'], inplace=True)

# drop the temporary calculated_year column
data.drop('calculated_year', axis=1, inplace=True)
```

Numerical Features: Numerical features which contained missing values in their features, filled with the mean of each features using SimpleImputer.

For categorical features, features that make up less than 1% of the data were clustered under what was termed 'Other'. It removes noise and improves the interpretability of categorical features on this step.

To handle outliers in a set of numerical features, the Interquartile Range (IQR) technique was used. For each feature, outliers were identified as values outside the range of $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$. These outliers have been removed from dataset to maintain more accuracy of the results to be obtained.

```
# removing outliers using IQR for numerical features
for feature in numerical_features:
    Q1 = data[feature].quantile(0.25)
    Q3 = data[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    data = data[(data[feature] >= lower_bound) & (data[feature] <= upper_bound)]
```

At the end, I removed the target (Price) from the list of numerical features to prevent unintended preprocessing that could impact model predictions.

To decrease the skewness and make the variance in the numerical features more stable the Power Transformer was used.

```
# applying Power Transformation to reduce skewness and handle noise
pt = PowerTransformer()
data[numerical_features] = pt.fit_transform(data[numerical_features])
```

The categorical features were processed using two encoding techniques to ensure they are represented numerically; **Target Encoding** and **One-Hot Encoding**. At the end, the dataset was split into training and testing data which 80 percent was used for training while the rest 20 percent was used for testing. In this model our target (Price) is `y` and `X` is defined as the combination of numerical and categorical features.

```
# define which columns should be encoded using which method
target_encoded_cols = [col for col in categorical_features if col not in ['fuel_type', 'vehicle_condition']]
one_hot_encoded_cols = ['fuel_type', 'vehicle_condition']

# create the preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('target', TargetEncoder(), target_encoded_cols),
        ('one_hot', OneHotEncoder(sparse_output=False), one_hot_encoded_cols)
    ],
    remainder='passthrough'
)

# split the data
y = data[target]
X = data[categorical_features + numerical_features]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

2. Automated Feature Selection

In order to increase the efficiency and interpretability of the model, automatic feature selection was carried out based on two widely used techniques: Recursive Feature Elimination with Cross-Validation (RFECV) and Sequential Feature Selection (SFS), both coupled with a linear regression estimator. These techniques enable identifying the most significant features that enhance predictive power while, simultaneously, minimizing redundancy and noise.

The RFECV approach recursively eliminated the least significant features and assessed model performance via 5-fold cross-validation. As can be seen from the respective plot (Figure 2.1), the R^2 score spiked as soon as over 6 features were added, levelling off after about 10. The optimal model had a test R^2 of 0.733 and a mean absolute error (MAE) of about 2834, showing good generalisation performance. The chosen features were a combination of categorical and numerical variables, notably: reg_code, standard_make, standard_model, and mileage.

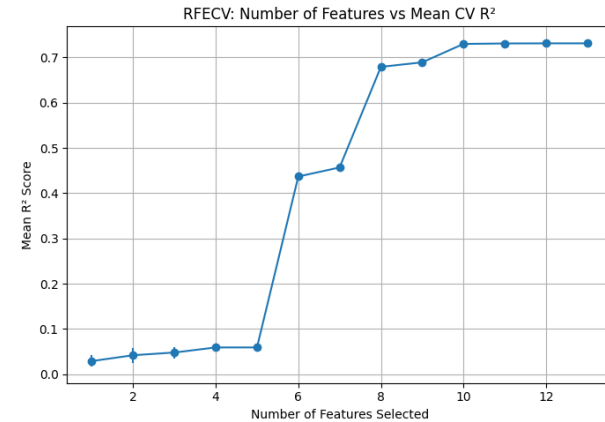


Figure 2.1

The evaluation involved implementation of SFS as a comparison framework to RFECV. The selected features generated through Figure 2.2 produced a rapid performance boost which sustained at a stable level. The metric results of SFS matched precisely with those of RFECV because both produced test R^2 of 0.733 along with test MAE of 2834.

```
# RFE + Linear Regression
regr_rfe = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("featsel", RFECV(LinearRegression(), step=1, cv=5)),
        ("regr", LinearRegression())
    ]
)

regr_rfe.fit(X_train, y_train)
```

```
Evaluation - RFECV:
RFECV - Train R²: 0.7308672002067917
RFECV - Test R²: 0.7331023622966192
RFECV - Train MAE: 2847.682079669637
RFECV - Test MAE: 2834.0103754512556
Selected Features by RFECV:
- reg_code
- standard_colour
- standard_make
- standard_model
- body_type
- fuel_type_Diesel
- fuel_type_Other
- fuel_type_Petrol
- fuel_type_Petrol Hybrid
- fuel_type_Petrol Plug-in Hybrid
- mileage
- year_of_registration
```

```
X_train_prep = preprocessor.transform(X_train)
X_test_prep = preprocessor.transform(X_test)

base_model = LinearRegression()

sfs = SFS(
    estimator=base_model,
    k_features='best',
    forward=True,
    floating=False,
    scoring='r2',
    cv=5,
    n_jobs=-1
)

sfs = sfs.fit(X_train_prep, y_train)
```

```
Selected Features by SFS:
- reg_code
- standard_colour
- standard_make
- standard_model
- body_type
- fuel_type_Diesel
- fuel_type_Other
- fuel_type_Petrol Hybrid
- fuel_type_Petrol Plug-in Hybrid
- mileage
- year_of_registration
Evaluation - SFS:
SFS - Train R²: 0.7308672002067919
SFS - Test R²: 0.7331023622966192
SFS - Train MAE: 2847.6820796696343
SFS - Test MAE: 2834.010375451253
```

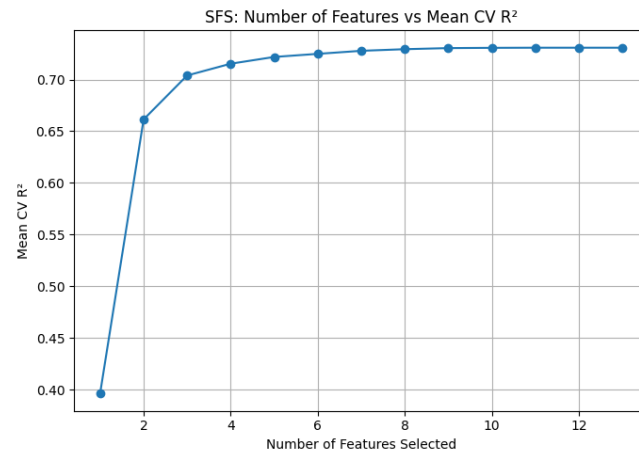


Figure 2.2

3. Tree Ensembles

This section implements the two ensemble methods Random Forest and XGBoost (Boosted Tree) for evaluation. Both models were trained on the same preprocessed dataset for consistent comparison.

Using all selected features during the previous step the Random Forest model was configured with 100 trees for training purposes. The training data results displayed a model performance with the R^2 value of 0.976 in Figure 3.1. During testing the R^2 measure decreased to 0.848 while the value of MAE rose from 720 on training to 1880 (Figure 3.2). The moderate level of overfitting conforms to expectations for standard Random Forest models due to their high complexity and unrestrained nature.

```
# Random Forest
regr_rf = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regr", RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1))
    ]
)

# training the model
regr_rf.fit(X_train, y_train)
```

```
Evaluation - Random Forest:
Train R²: 0.9764470046748598
Test R²: 0.8477139165368779
Train MAE: 720.1759341872345
Test MAE: 1880.5891658424287
```

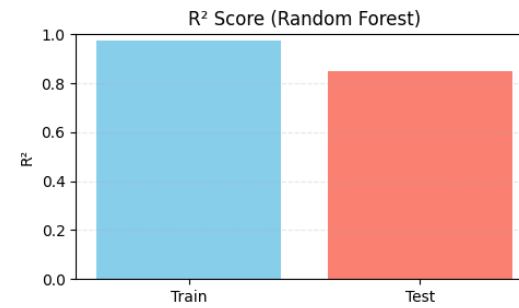


Figure 3.1

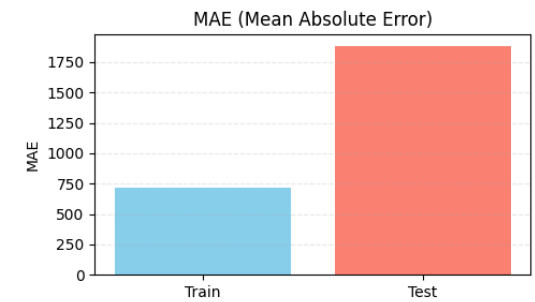


Figure 3.2

XGBoost was trained using 100 boosting rounds and a squared error loss objective. It generalises slightly better to the test data with the R^2 of 0.861 and a test MAE of 1860 (Figures 3.3 and 3.4). The smaller gap between training and test R^2 scores (0.864 vs. 0.861) and MAEs (1835 vs. 1860) reflect better regularisation and better generalisation performance, with XGBoost now being a more well-rounded option compared to Random Forest in this context.

```
# XGBoost
regr_xgb = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regr", XGBRegressor(objective='reg:squarederror', n_estimators=100, random_state=42))
    ]
)

# training
regr_xgb.fit(X_train, y_train)
```

```
Evaluation - XGBoost:
Train R²: 0.8643288144468007
Test R²: 0.8605298351228385
Train MAE: 1835.1149372988516
Test MAE: 1860.173054328991
```

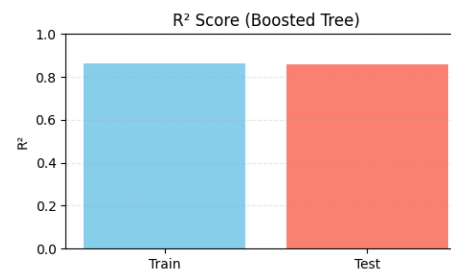


Figure 3.3



Figure 3.4

The significant training results from Random Forest indicate an overfitting tendency which appears in the model's generalisation gap. XGBoost demonstrated a consistent model performance between both sets of data. Both ensemble models demonstrate effectiveness for structured data according to the results while boosting models exhibit greater robustness.

4. Ensemble of Tree Ensembles

To further enhance predictive performance and model robustness, an ensemble technique known as stacking was implemented. Using stacking technique multiple diverse base models receive their predictions as input to a meta-model that tries to overcome model limitations.

In this section I implemented two tree-based regression learners: Random Forest with 50 estimators and XGBoost with 50 boosting rounds using hist tree optimisation. The simple linear regression model functioned as the meta-learner to combine predictions from the base models. The stacking model operated at float32 precision while disabling parallel execution (`n_jobs=1`) to prevent memory issues.

```
numeric_cols = X_train.select_dtypes(include=['float64', 'int64']).columns.tolist()

# transforming to float32
X_train[numeric_cols] = X_train[numeric_cols].astype(np.float32)
X_test[numeric_cols] = X_test[numeric_cols].astype(np.float32)

# base model/learner
base_learners = [
    ('rf', RandomForestRegressor(n_estimators=50, random_state=42, n_jobs=1)),
    ('xgb', XGBRegressor(
        objective='reg:squarederror',
        n_estimators=50,
        tree_method='hist',
        random_state=42,
        n_jobs=1
    ))
]

meta_model = LinearRegression()

stacking_model = StackingRegressor(
    estimators=base_learners,
    final_estimator=meta_model,
    cv=3,
    n_jobs=1
)

regr_stack = Pipeline([
    ("preprocessor", preprocessor),
    ("regr", stacking_model)
])

# training
regr_stack.fit(X_train, y_train)
```

The test R^2 score from the stacked model reached 0.861 at the same level as XGBoost. The stacked model demonstrated a test MAE result of 1838 while maintaining optimal performance and generalisation compared to individual Random Forest and XGBoost models. The stacking method successfully took advantage of both approaches as it minimised Random Forest overfitting and produced smaller training MAE than XGBoost (refer to Figures 4.1 and 4.2).

The R^2 scores presented in Figure 4.1 show the comparison between models. The stacking model demonstrates performance near Random Forest and XGBoost metrics on test data while maintaining a relatively low overfitting gap same as XGBoost. The MAE values presented in Figure 4.2 indicate that stacking performed a little bit better than other models on test data. In my opinion, XGBoost is more stable than the other models.

```
Evaluation - Stacking:
Train R²: 0.9240898186304256
Test R²: 0.8615852268128072
Train MAE: 1367.7364303939576
Test MAE: 1838.9117136392601
```

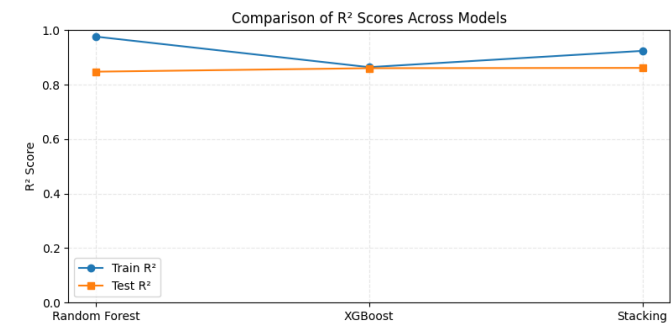


Figure 4.1

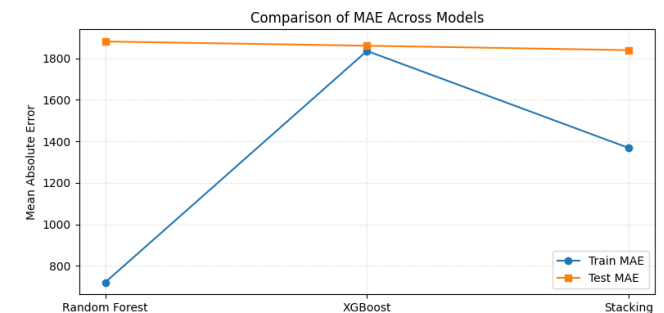


Figure 4.2

5. Feature Importance

Complex model interpretation and feature analysis for price prediction required the application of permutation-based and model-based importance techniques. The analysis utilised permutation importance with Random Forest and XGBoost together with SHAP values extracted from the XGBoost model.

Model accuracy depends on each feature through permutation importance which calculates the performance reduction from random feature value shuffling. [Standard_make](#) stood as the most influential variable based on results displayed in Figures 5.1 and 5.2 when evaluated by Random Forest and XGBoost. Both models demonstrate similar levels of importance for the target variable (price) which confirms their connexion to the outcome.

Standard_make demonstrated the highest predictive power in the Random Forest model because its mean R^2 value dropped by more than 0.6 when permuted data was used. XGBoost displayed an equally distributed predictive model that used mileage and year_of_registration in addition to other variables for effective forecasting.

```
# fit to get feature names
preprocessor.fit(X_train, y_train)
onehot = preprocessor.named_transformers_['one_hot']
onehot_names = list(onehot.get_feature_names_out(one_hot_encoded_cols))
transformed_feature_names = target_encoded_cols + onehot_names + numerical_features

# Random Forest

result_rf = permutation_importance(
    regr_rf.named_steps['regr'],
    X_test_prep,
    y_test,
    n_repeats=10,
    random_state=42
)

feature_names_used_rf = transformed_feature_names[:len(result_rf.importances_mean)]
perm_importances_rf = pd.Series(result_rf.importances_mean, index=feature_names_used_rf)
```

```
# XGBoost
result_xg = permutation_importance(
    regr_xgb.named_steps['regr'],
    X_test_prep,
    y_test,
    n_repeats=10,
    random_state=42
)

feature_names_used_xgb = transformed_feature_names[:len(result_xg.importances_mean)]
perm_importances_xg = pd.Series(result_xg.importances_mean, index=feature_names_used_xgb)
```

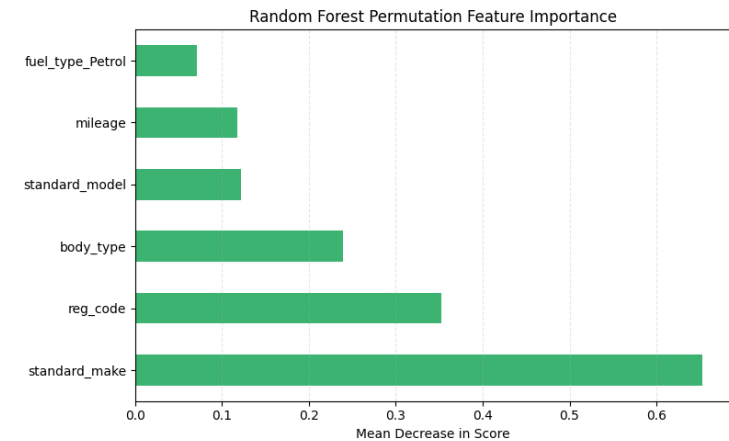


Figure 5.1

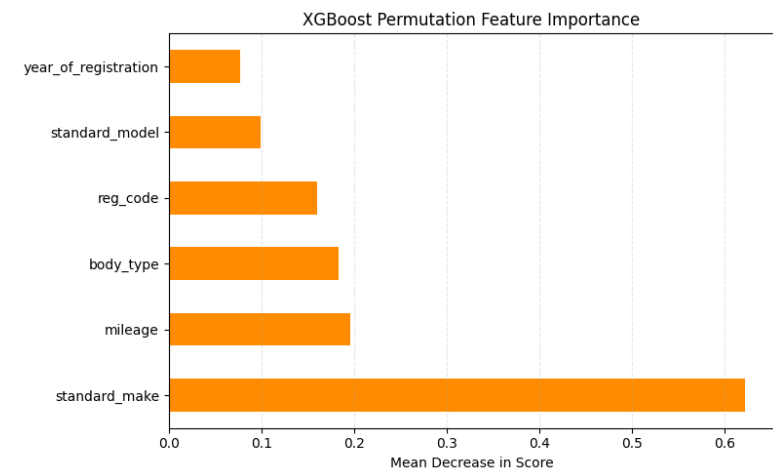


Figure 5.2

The XGBoost model received feature importance validation through SHAP analysis. The SHAP summary plot (Figure 5.3) displays how feature impacts spread throughout the dataset. Standard_make and reg_code emerged once more as dominant predictive variables. The colour scale reveals that reg_code and mileage values at higher levels generate lower predicted prices but specific makes produce substantial model output.

```
import shap
# get the feature names
feature_names = transformed_feature_names[:X_train_prep.shape[1]]
# create a DataFrame from the processed data
X_df = pd.DataFrame(X_train_prep, columns=feature_names)
# extract the raw XGBoost model
xgb_model = regr_xgb.named_steps['regr']
# create the SHAP explainer
explainer = shap.Explainer(xgb_model)
shap_values = explainer(X_df)
# Summary Plot
shap.summary_plot(shap_values, X_df, plot_type="dot", max_display=10)
```

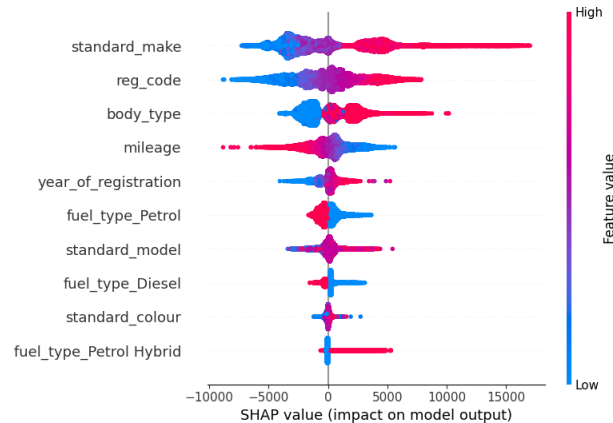


Figure 5.3 (XGBoost)

6. SHAP/PDP Model Explanations

A random sample of 100 observations was used to enable visualisations and improve runtime efficiency when applying SHAP to large datasets, especially for Random Forest. Standard_make, reg_code, body_type and mileage emerged as primary predictors which influenced the predicted price values throughout the sampled observations according to the SHAP summary plots (Figures 6.1 and 6.2). The feature importance rankings among XGBoost and Random Forest algorithms approximately matched which validated the reliability of chosen features in the selection process.

SHAP waterfall plots were used to generate visualisations that explored individual prediction explanations on a local level (Figures 6.3 and 6.4). The visualisations dissect how each feature element affects one specific example. The price prediction either rose or decreased substantially based on the values assigned to features body_type and standard_make.

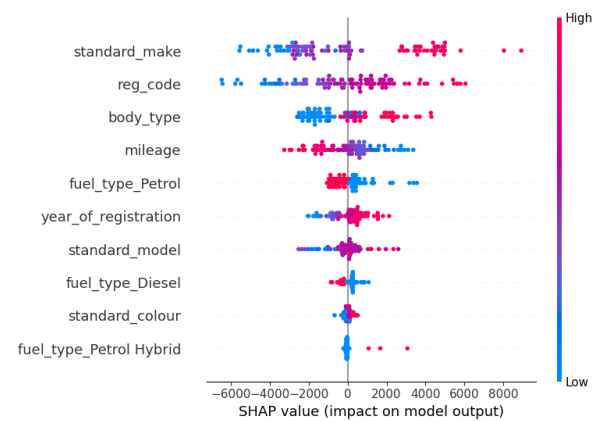


Figure 6.1 (XGBoost)



Figure 6.2 (Random Forest)

```
# Waterfall-plots for XGBoost
print("Waterfall plots for XGBoost:")
for i in range(1):
    print(f"Sample {i}")
    shap.plots.waterfall(shap_values_xgb[i], max_display=10)

# Waterfall-plots for Random Forest
print("Waterfall plots for Random Forest:")
for i in range(1):
    print(f"Sample {i}")
    shap.plots.waterfall(shap_values_rf[i], max_display=10)
```

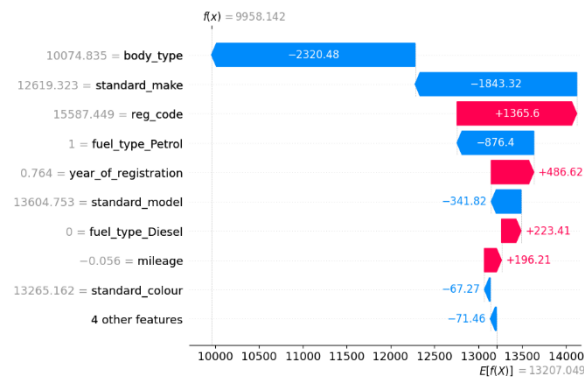


Figure 6.3 (XGBoost)

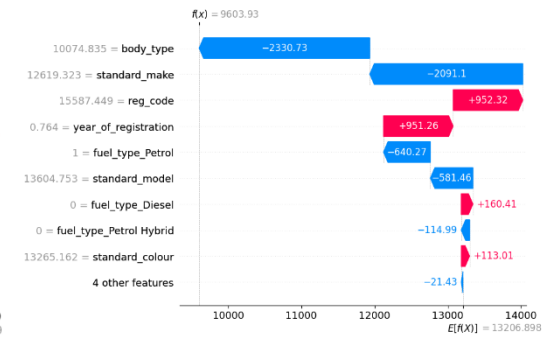


Figure 6.4 (Random Forest)

To increase the model interpretability and to get a sense of the influence of particular features on the predicted car prices, Partial Dependence Plots (PDPs) and Individual Conditional Expectation (ICE) plots were created for two key continuous features: mileage and year_of_registration. The plots enable the detection of the marginal impact of one feature on the target, averaged across the distribution of all the other features.

Figures 6.5 and 6.6 display the PDP curves for XGBoost followed by Random Forest. The plots show uniform patterns that exist throughout both model types:

- The predicted car price shows a regular decline when mileage amounts increase.
- The model predicts rising prices when new cars are added to the dataset (increased year_of_registration value).

The observed trends in the data support the correct prediction of models which follow standard domain knowledge principles.

```
from sklearn.inspection import PartialDependenceDisplay

# features we want to investigate
features_to_plot = ['mileage', 'year_of_registration']

# XGBoost
PartialDependenceDisplay.from_estimator(
    regr_xgb,
    X_test,
    features=features_to_plot,
    feature_names=X.columns,
    grid_resolution=50,
    kind='average'
)

plt.tight_layout()
plt.show()
```

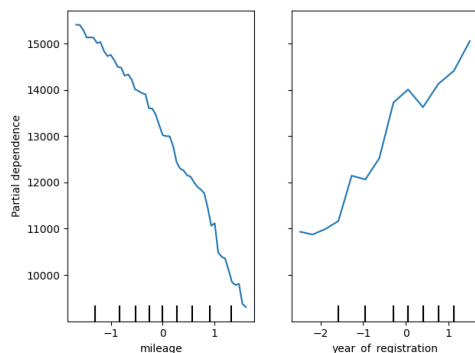


Figure 6.5 (XGBoost)

```
# Random Forest
PartialDependenceDisplay.from_estimator(
    regr_rf,
    X_test,
    features=features_to_plot,
    feature_names=X.columns,
    grid_resolution=50,
    kind='average'
)

plt.tight_layout()
plt.show()
```

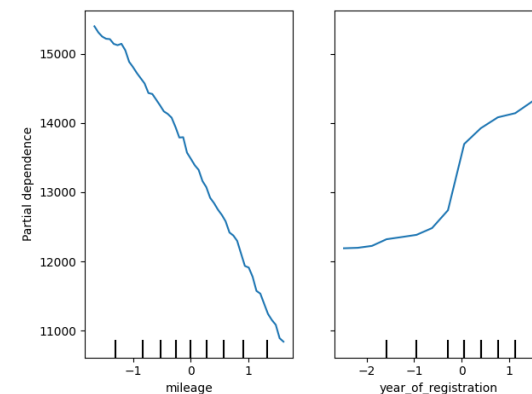


Figure 6.6 (Random Forest)

Feature effects were analysed at the individual level by overlaying ICE curves on PDP lines in Figures 6.7 and 6.8. The PDP lines show average trends but individual ICE lines display specific instance variations. The ICE lines from XGBoost model had denser and smoother patterns than Random Forest's curves which indicated improved stability in response across different test samples. Mileage demonstrates steadier effects compared to year_of_registration which displays larger variations in both models.

```
PartialDependenceDisplay.from_estimator(
    regr_xgb,
    X_test,
    features=features_to_plot,
    feature_names=X.columns,
    kind='both', # showing both ICE and PDP
    grid_resolution=50,
    pd_line_kw={'color': 'red'}, # red average line (PDP)
    ice_lines_kw={'color': 'blue', 'alpha': 0.1} # thin blue ICE lines with transparency
)

plt.suptitle("XGBoost - Partial Dependence + ICE", fontsize=14)
plt.tight_layout()
plt.show()
```

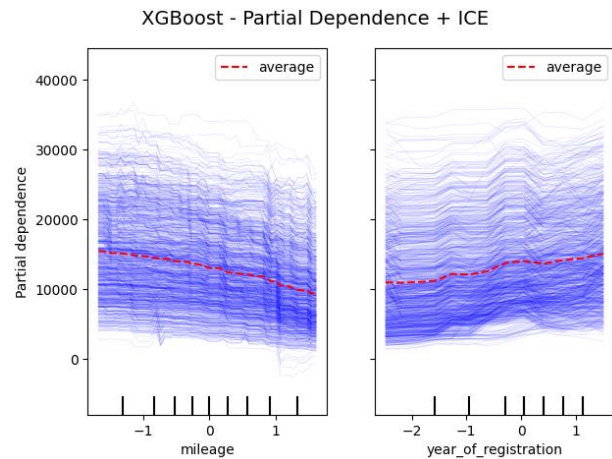


Figure 6.7

```
PartialDependenceDisplay.from_estimator(
    regr_rf,
    X_test,
    features=features_to_plot,
    feature_names=X.columns,
    kind='both',
    grid_resolution=50,
    pd_line_kw={'color': 'green'},
    ice_lines_kw={'color': 'gray', 'alpha': 0.15}
)

plt.suptitle("Random Forest - Partial Dependence + ICE", fontsize=14)
plt.tight_layout()
plt.show()
```

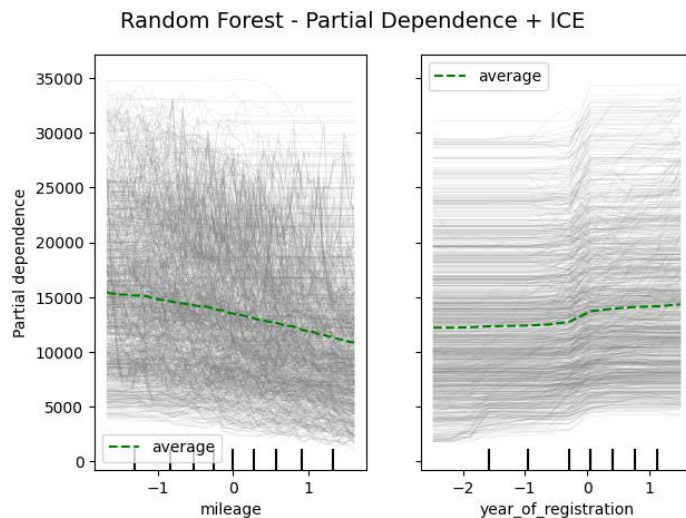


Figure 6.8

7. Dimensionality Reduction, Linear

Principal Component Analysis (PCA) became the method to decrease the high-dimensional dataset's size while achieving better interpretability and computational efficiency. PCA reshapes the feature space through orthogonal components which organise themselves according to their ability to explain data variations. The figure 7.1 presents the total explained variance distribution across principal components. A significant portion of the information can be maintained through the initial four components which together explain more than 95% of total variance. This result justifies using PCA as a preprocessing step for potential dimensionality-sensitive models such as K-Means or visualisation purposes.

```
from sklearn.decomposition import PCA

# applying PCA
pca = PCA()
X_train_pca = pca.fit_transform(X_train_prep)

# plot cumulative explained variance
explained_variance = np.cumsum(pca.explained_variance_ratio_)

plt.figure(figsize=(8, 4))
plt.plot(explained_variance, marker='o')
plt.title("Cumulative Explained Variance by PCA Components")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")
plt.grid(True, linestyle='--', alpha=0.4)
plt.axhline(0.95, color='red', linestyle='--', label='95% Variance Threshold')
plt.legend()
plt.tight_layout()
plt.show()
```

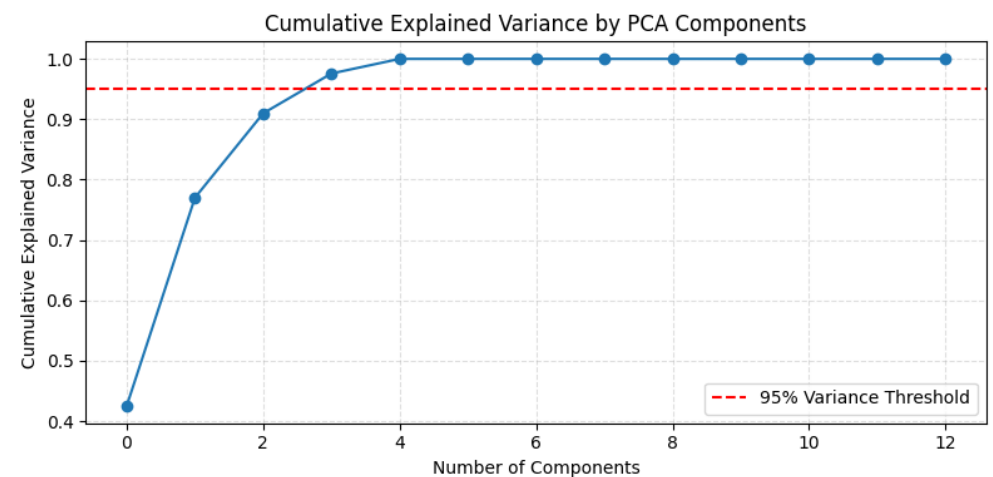


Figure 7.1

PCA analysis using `n_components=2` produced data visualisation in two dimensions which presented price quartiles (Q1–Q4) through colour coding. As shown in Figure 7.2, although the clusters are not linearly separable, there is a mild spatial pattern in how price groups are distributed along the principal components. The highest price group (Group 4) gathers into tight clusters which occupy certain areas of the plot while the lowest price group (Group 1) is more dispersed.

```
# applying PCA with 2 components
pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_train_prep)

# categorise y into price bins (like 4 groups)
price_bins = pd.qcut(y_train, q=4, labels=[f"Group {i}" for i in range(1, 5)])

# create a DataFrame for plotting
df_pca = pd.DataFrame(X_pca_2d, columns=['PC1', 'PC2'])
df_pca['Price Group'] = price_bins.values

# plot a PairPlot
sns.pairplot(df_pca, hue='Price Group', palette='viridis')
plt.suptitle("PCA Projection (2D) Coloured by Price Groups", y=1.02)
plt.tight_layout()
plt.show()
```

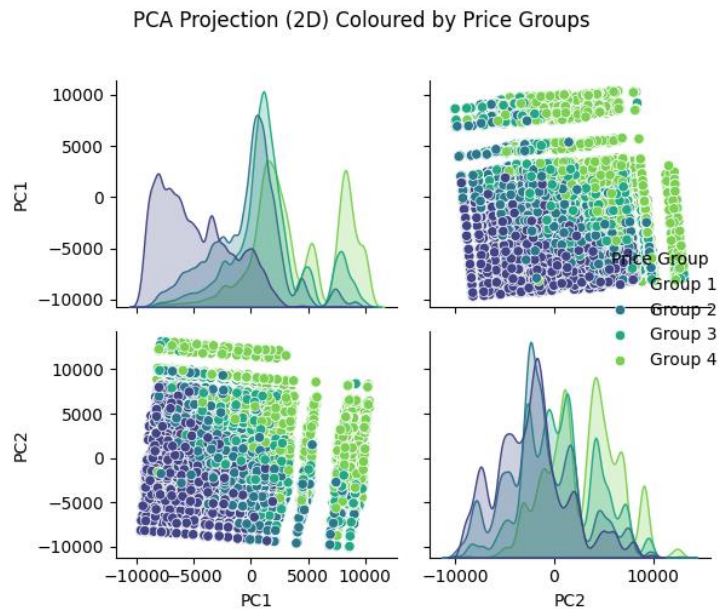


Figure 7.2

8. Dimensionality Reduction, Non-Linear

To capture complex, non-linear structures within the dataset, Isomap was employed as a manifold learning technique. Instead of linear projections (PCA), Isomap uses neighbourhood graph computations to maintain geodesic sample distances which enables better discovery of intrinsic data geometry. A subset of preprocessed training samples totaling 5000 examples was used to execute Isomap with `n_components=2` and `n_neighbors=10` parameters. Figure 8.1 visualises a 2D projection where colour schemes represent price quartiles while showing distinct clusters especially for higher-price vehicles (Group 4), which tend to occupy distinct areas of the embedding space. The results demonstrate that Isomap detects non-linear relations between variables that affect car price values.

```
from sklearn.manifold import Isomap
from sklearn.preprocessing import StandardScaler

# sample the data to reduce computational load
X_sample, _, y_sample, _ = train_test_split(X_train_prep, y_train, train_size=5000, random_state=42)

# categorise prices into 4 groups
price_bins_iso = pd.qcut(y_sample, q=4, labels=[f"Group {i}" for i in range(1, 5)])

# applying Isomap
isomap = Isomap(n_neighbors=10, n_components=2)
X_iso = isomap.fit_transform(X_sample)

# creating a DataFrame for plotting
df_iso = pd.DataFrame(X_iso, columns=['Dim1', 'Dim2'])
df_iso['Price Group'] = price_bins_iso.values

# plot the projection
sns.pairplot(df_iso, hue='Price Group', palette='coolwarm')
plt.suptitle("Isomap Projection (2D) Coloured by Price Groups", y=1.02)
plt.tight_layout()
plt.show()
```

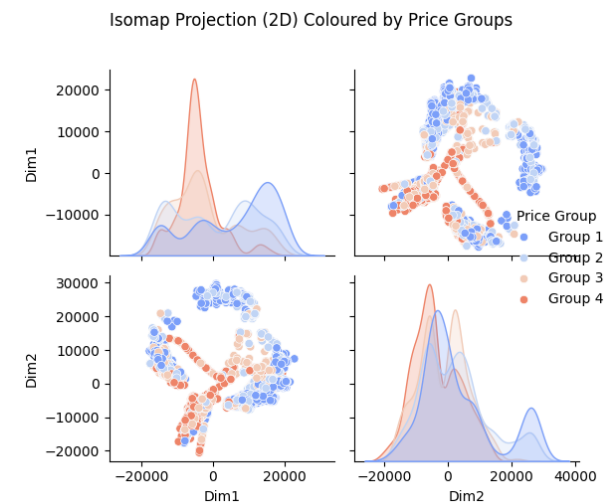


Figure 8.1

A different selection of 3000 samples was applied to generate Figure 8.2 where the Isomap projection showed continuous price values. Visual analysis of the colour gradient demonstrates distinct price variations that occur in particular spatial directions of the 2D space. The pattern of data distribution supports this idea about meaningful underlying non-linear relations between features and the target variable.

```
# sample the data
X_sample, _, y_sample, _ = train_test_split(X_train_prep, y_train, train_size=3000, random_state=42)

# applying Isomap
isomap = Isomap(n_components=2, n_neighbors=15)
X_iso = isomap.fit_transform(X_sample)

# plot the projection coloured by actual price
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_iso[:, 0], X_iso[:, 1], c=y_sample, cmap='viridis', s=20)
plt.title("Isomap Projection Coloured by Actual Price")
plt.xlabel("Isomap Component 1")
plt.ylabel("Isomap Component 2")
plt.colorbar(scatter, label="Price")
plt.tight_layout()
plt.show()
```

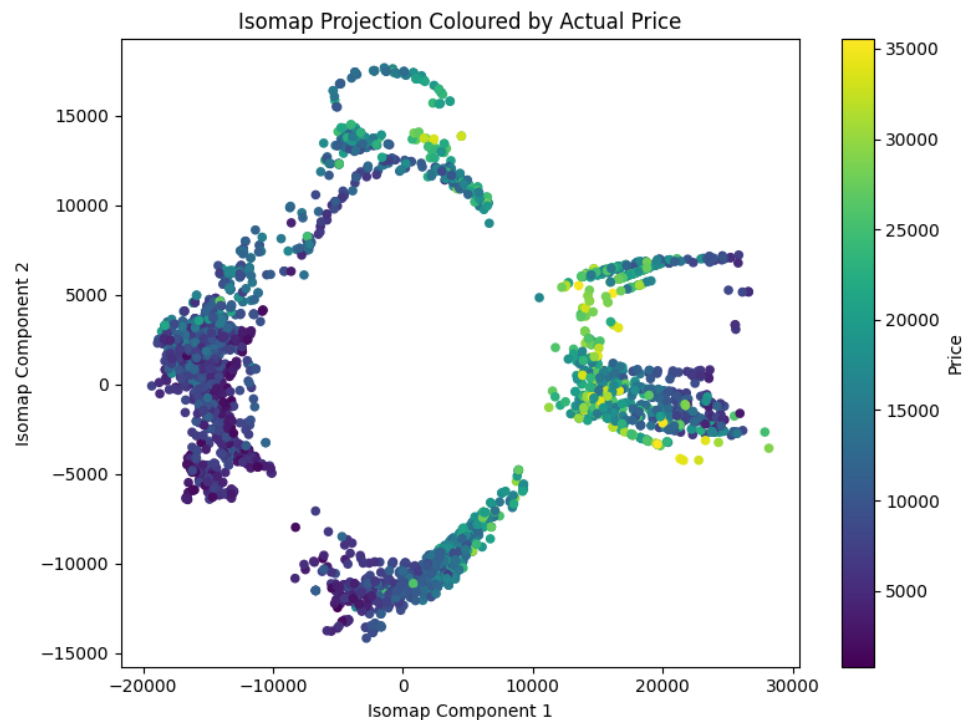


Figure 8.2

9. Polynomial Regression

To analyse of a basic non-linear model, a second-degree Polynomial Regression was trained using numerical features exclusively. The model pipeline included polynomial feature expansion followed by a standard linear regression estimator.

The model implementation with non-linear interactions achieved weak performance because it produced a test R^2 of 0.388 and an MAE of 4458. The test and training score line-up closely because the model handled the data structure imperfectly while maintaining stability between results sets. The results demonstrate that polynomial regression works well on small-scale problems but it cannot handle high-dimensional and complex datasets.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

# selecting only numerical features
X_num_train = X_train[numerical_features]
X_num_test = X_test[numerical_features]

# creating a pipeline: polynomial feature generation + linear regression model
poly_model = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    LinearRegression()
)

# training the model
poly_model.fit(X_num_train, y_train)

# make predictions
y_pred_train = poly_model.predict(X_num_train)
y_pred_test = poly_model.predict(X_num_test)

# evaluation
print("Evaluation - Polynomial Regression (Degree 2)")
print("Train R²:", r2_score(y_train, y_pred_train))
print("Test R²:", r2_score(y_test, y_pred_test))
print("Train MAE:", mean_absolute_error(y_train, y_pred_train))
print("Test MAE:", mean_absolute_error(y_test, y_pred_test))
```

```
Evaluation - Polynomial Regression (Degree 2)
Train R²: 0.38644970169462123
Test R²: 0.3879708951160401
Train MAE: 4459.474041406661
Test MAE: 4457.763263346253
```

10. Clustering for Feature Engineering

K-Means clustering was applied to preprocessed training data to identify how vehicles can group according to their characteristics through this section. The Elbow Method provided data to select the best number of clusters from Figure 10.1. Analysis of the plot indicated a "knee" point at k=5 thus leading to selection of this value for final clustering. The preprocessing stage included target encoding combined with one-hot encoding and normalisation through PowerTransformer and StandardScaler.

```
# split sample train/test data
X_train_sample, _, y_train_sample, _ = train_test_split(X_train, y_train, train_size=10000, random_state=42)

# preprocessing
preprocessor_cluster = ColumnTransformer(
    transformers=[
        ('target', TargetEncoder(), target_encoded_cols),
        ('one_hot', OneHotEncoder(sparse_output=False), one_hot_encoded_cols),
        ('num', PowerTransformer(), numerical_features)
    ]
)

X_proc = preprocessor_cluster.fit_transform(X_train_sample, y_train_sample)

# normalisation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_proc)
```

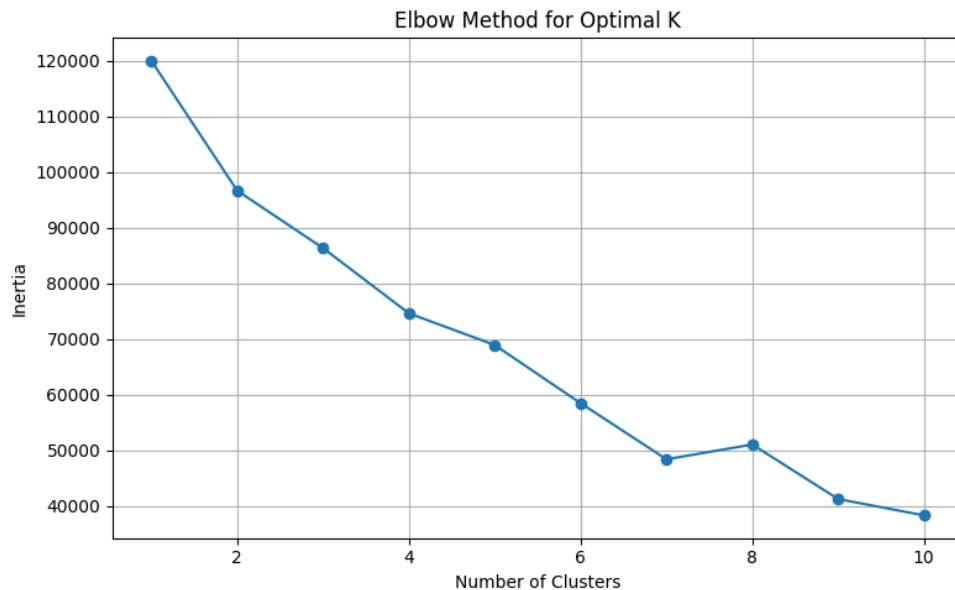


Figure 10.1

The PCA was used to transform the features into two components which highlighted cluster distribution. The executed 2D scatter plot (Figure 10.2) provided evidence of five distinct clusters which indicates that KMeans successfully extracted data structural patterns. Four clusters are more visible on the plot as you can see. To fixing this problem, I tried to set four clusters instead of five and the result was three visible clusters, so I decided to stay with five clusters which is the best number for this dataset in my opinion, and Figure 10.1 was confirmed it.

```
# applying KMeans with a specified number of clusters (like 5)
optimal_k = 5
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init='auto')
cluster_labels = kmeans_final.fit_predict(X_scaled)

# reduce dimensions to 2 components using PCA for visualisation
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# create a DataFrame for plotting
df_clusters = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df_clusters['Cluster'] = cluster_labels

# plot scatterplot with colours based on clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df_clusters, x='PC1', y='PC2', hue='Cluster', palette='Set2', s=70)
plt.title("KMeans Clustering (k=5) - PCA Projection")
plt.grid(True)
plt.tight_layout()
plt.show()
```

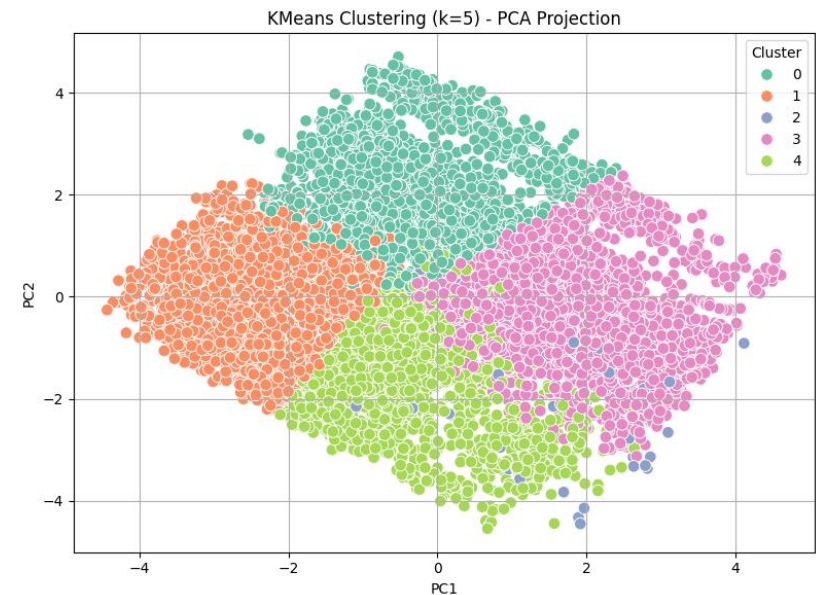


Figure 10.2